

CS 162

Computer Architecture

University of California, Riverside

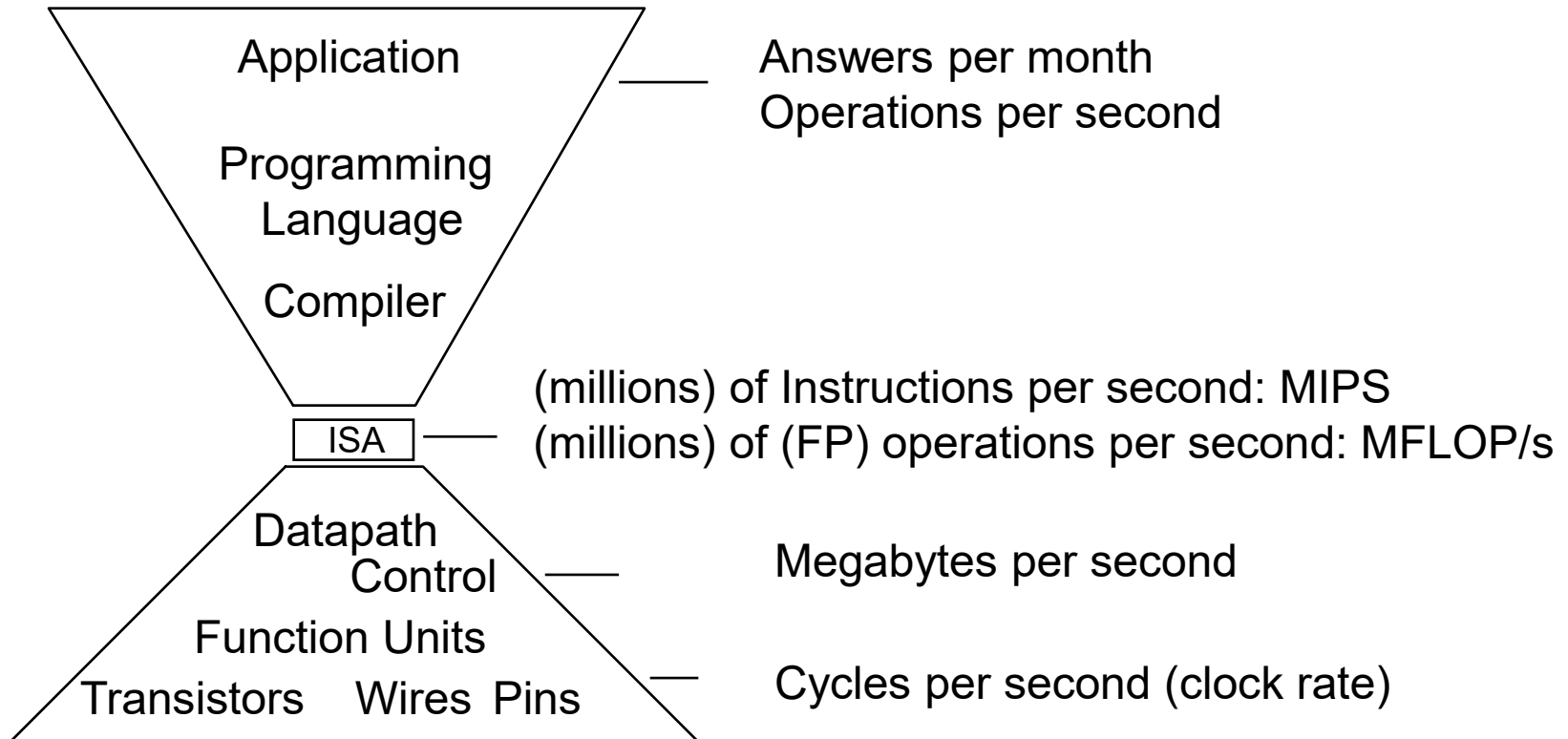
Bingyao Li
<http://libingyao.github.io/>

Today's Class



- Lecture
 - How we measure performance

Performance (§1.8)



Measuring Performance

- **Time to run the task (latency)**
 - Execution time, response time, CPU time, ...
- **Tasks per day, hour, week, sec, ns, ...**
 - Throughput, bandwidth

Performance measurement Tools

- Hardware prototypes : Cost, delay, area, power estimation
- Simulation (many levels, ISA, RTL, Gate, Circuit, ...)
- Benchmarks (Kernels, toy programs, synthetic), Traces, Mixes
- Analytical modeling and Queuing Theory

Relative Performance

- To maximize performance, need to minimize execution time

$$\text{performance}_x = 1 / \text{execution_time}_x$$

If computer X is n times faster than computer Y, then

$$\frac{\text{performance}_x}{\text{performance}_y} = \frac{\text{execution_time}_y}{\text{execution_time}_x} = n$$

- Decreasing response time almost always improves throughput

Relative Performance Example

- If computer A runs a program in 10 seconds and computer B runs the same program in 15 seconds, how much faster is A than B?

How to Summarize Performance



- Arithmetic mean (weighted arithmetic mean)
 - ex: tracks execution time: $\sum_{i=1}^n \frac{T_i}{n}$ or $\sum_{i=1}^n W_i * T_i$
- Harmonic mean (weighted harmonic mean) of rates
 - ex: track MFLOPS: $\frac{n}{\sum_{i=1}^n \frac{1}{Rate_i}}$
- Normalized execution time is handy for scaling performance (e.g., X times faster than Pentium 4)
- Geometric mean $\Rightarrow \sqrt[n]{\prod_{i=1}^n execution_ratio_i}$
where the execution ratio is relative to a reference machine

Performance Evaluation



- Good products created when we have:
 - Good benchmarks
 - Good ways to summarize performance
- For better or worse, benchmarks shape a field.
- Execution time and power are the main measure of computer performance!
- Reproducibility is important (should provide details of experiments)

Benchmarks



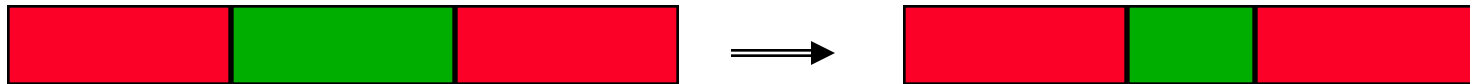
- SPEC2017: Standard Performance Evaluation Corporation
- PARSEC: Princeton Application Repository for Shared-Memory Computers
- MediaBench: Multimedia and embedded applications
- Transaction processing- TPC-C, SPECjbb
- Embedded Microprocessor Benchmark Consortium – EEMBC: Networking, telecom, digital cameras, cellular phones, ...
- Stanford parallel benchmarks: For parallel architecture and shared memory multiprocessors
- NAS: For massively parallel processor systems
- Rodinia: for GPU applications
- AI/DNN: MLperf, PMLB, MLbench

Amdahl's Law (§ 1.9)

Speedup due to some enhancement E:

$$Speedup_{overall} = \frac{ExTime_{withoutE}}{ExTime_{withE}} = \frac{Performance_{withE}}{Performance_{withoutE}}$$

Suppose that enhancement E accelerates a fraction of the task by a factor S, and the remainder of the task is unaffected



$$\frac{ExTime_{withE}}{ExTime_{withoutE}} = (1 - fraction_{enhanced}) + \frac{fraction_{enhanced}}{S}$$

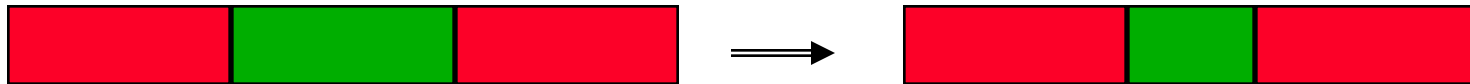
Example: Floating point instructions can be improved to run 2X; but only 10% of actual instructions are FP. What is the overall speedup?

Amdahl's Law (§ 1.9)

Speedup due to some enhancement E:

$$Speedup_{overall} = \frac{ExTime_{withoutE}}{ExTime_{withE}} = \frac{Performance_{withE}}{Performance_{withoutE}}$$

Suppose that enhancement E accelerates a fraction of the task by a factor S, and the remainder of the task is unaffected



$$\frac{ExTime_{withE}}{ExTime_{withoutE}} = (1 - fraction_{enhanced}) + \frac{fraction_{enhanced}}{S}$$

Example: How much faster must the multiplier be to get a 2x performance improvement overall if multiples account for 20 seconds of the 100 second run time?

Computing CPU time

$$\text{Average Cycles per Instruction (CPI)} = \sum_{j=1}^n CPI_j * F_j$$

Where CPI_j is the number of cycles needed to execute instructions of type j , and F_j is the percentage (fraction) of instructions that are of type j .

Example: Base Machine (Reg / Reg)

Op	Freq	Cycles	$CPI_j * F_j$	(% Time)
ALU	50%	1	.5	(33%)
Load	20%	2	.4	(27%)
Store	10%	2	.2	(13%)
Branch	20%	2	.4	(27%)
			1.5	

Typical Mix

$$\text{Instructions Per Cycle (IPC)} = 1 / CPI$$

$$CPU\ time = Cycle\ time * \sum_{j=1}^n (CPI_j \times I_j)$$

Where I_j is the number of instructions of type j , and

Cycle time is the inverse of the *clock rate*.

$$CPU\ time = \text{total \# of instructions} \times CPI \times Cycle\ time$$

Example: For some programs,

Machine A has a clock cycle time of 10 ns. and a CPI of 2.0

Machine B has a clock cycle time of 20 ns. and a CPI of 1.2

What machine is faster for this program, and by how much?

$$CPI \times Cycle\ time$$

A Simple Performance Tradeoff Example



Op	Freq	CPI _i	Freq x CPI _i
ALU	50%	1	.5
Load	20%	5	1.0
Store	10%	3	.3
Branch	20%	2	.4
Average (Effective) CPI			$\Sigma = 2.2$

.5 .5 .25

.4 1.0 1.0

.3 .3 .3

.4 .2 .4

1.6 2.0 1.95

- How much faster would the machine be if a better data cache reduced the average load time to 2 cycles?

CPU time new = 1.6 x IC x CC so 2.2/1.6 means 37.5% faster

- How does this compare with using branch prediction to shave a cycle off the branch time?

CPU time new = 2.0 x IC x CC so 2.2/2.0 means 10% faster

- What if two ALU instructions could be executed at once?

CPU time new = 1.95 x IC x CC so 2.2/1.95 means 12.8% faster

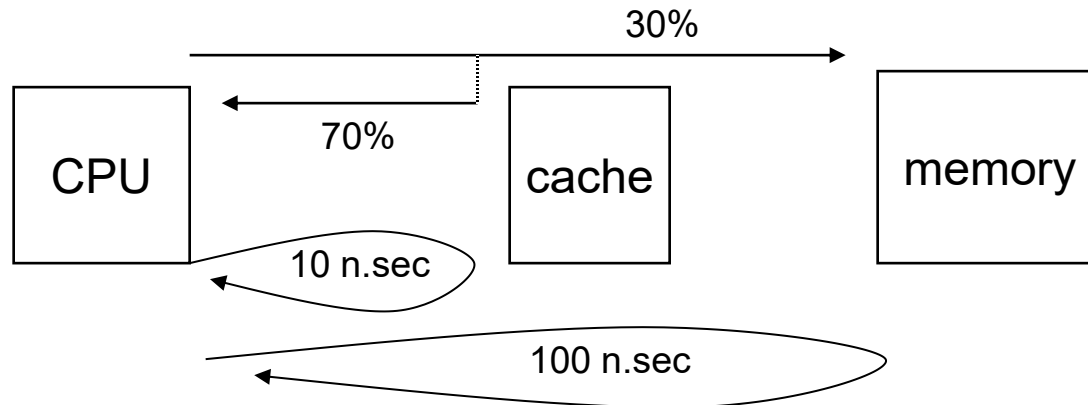
Aspects of CPU Performance

$$CPU_time = \frac{Seconds}{program} = \frac{Instructions}{program} \times \frac{Cycles}{Instructions} \times \frac{Seconds}{Cycle}$$

	Inst Count	CPI	Clock Rate
Program	X		
Compiler	X	X	
Inst. Set.	X	X	
Organization		X	X
Technology			X

Improving CPI using caches

An example:



What is the improvement (speedup) in memory access time?

Caching works because of the principle of locality:

- Locality found in memory access instructions
 - Temporal locality: if an item is referenced, it will tend to be referenced again soon
 - Spatial locality: if an item is referenced, items whose addresses are close by tend to be referenced soon
- 90/10 locality rule
 - A program executes about 90% of its instructions in 10% of its code
- We will look at how this principle is exploited in various microarchitecture techniques